

AWS Certified Generative AI Developer — Professional

The Use Case

- Multinational insurer, 50,000+ claims monthly across auto, home, health and commercial lines
- **Problems:** lengthy processing times, inconsistent adjudication decisions, manual document bottlenecks, poor customer satisfaction
- **Results:** processing down from 12-15 days to 1-3 days, 42% better adjudication consistency, 78% document automation, 35% fewer appeals, 47% higher satisfaction

Documentation error in the source notes: they say " `text/event-stream` for server-side encryption [SSE]". Here **SSE means Server-Sent Events**, not encryption. `text/event-stream` is the streaming content type.

This task overlaps heavily with 2.1 and 2.4. What is **genuinely new**: token windowing and prompt compression, Amplify UI and accessibility, Amazon Q Business and Q Developer, Bedrock Data Automation, prompt registries, and synthetic monitoring.

1. FM API Interfaces

Token windowing

```
track token usage
-> approaching the limit (typically 4K-32K)
-> RETAIN critical context
-> REMOVE less relevant content
```

Transcript manages context within an 8,000-token limit. This is Task 1.6's context windowing made concrete with token counting.

Request chunking and recursive summarization

```
chunk 1 processed      -> summary
chunk 2 + summary      -> processed -> new summary
chunk 3 + summary      -> ...
```

Large prompts are split into segments, and **each chunk carries forward the distilled context of the previous ones**, preserving coherence across boundaries.

Tiered retry — different backoff per error type

```
429 (throttling) -> IMMEDIATE retry, jitter 100-300 ms
5xx (server)     -> EXPONENTIAL backoff from 500 ms
3-5 failures in a 30-second window -> CIRCUIT BREAK
```

Why the difference: a 429 means capacity is momentarily unavailable and will return shortly — retry almost immediately. A 5xx means something is broken and needs longer to recover. Transcript uses a circuit breaker after **four** failures.

Timeouts at four levels

```
Client
API Gateway    -> 30-60s standard, 120s+ complex generation
Lambda
Model invocation
```

These must be aligned. If Lambda's timeout is shorter than API Gateway's, Lambda dies first and the gateway never learns why. Add client-side progress indicators as requests approach the threshold.

WebSocket settings and content filtering

- Idle timeouts of **10-30 minutes** for long-running GenAI tasks (transcript uses 20); ping/pong every 30-60 seconds
- **Content filtering middleware** validates **both input prompts and model responses** against policy, logs violations, and falls back when content is rejected — the Guardrails idea from Task 1.6 as an explicit layer
- Response templates set the right content type: `text/event-stream` for SSE, `application/json` for chunked JSON

2. Accessible AI Interfaces

Everything so far has been backend. This section is the **front end**.

Amplify UI with progressive enhancement

Pattern	Purpose
Skeleton screens	While the model generates, show the <i>shape</i> of the coming content (grey blocks) rather than a spinner — it reads as progress
Error states	Explain a failed request clearly instead of leaving a blank screen

Progressive enhancement means basic functionality works everywhere, with richer behaviour where the client supports it.

Progressive rendering and client-side caching

```
Render updates: every 50-200 ms
Cache hit when input similarity: 85-95%
plus time-based expiration
```

Similarity-based caching, not exact match — people ask the same thing in different words. Note this is a tighter interval than Task 2.4's 300-500 ms debounce, because that concerned network efficiency while this concerns rendering.

Accessibility

- **ARIA attributes**, keyboard navigation, screen reader compatibility for AI-generated content
- Proper loading states and progress indicators
- **Why this is unusually important for GenAI:** content arrives incrementally, so screen readers must be told the region is updating (ARIA live regions)

OpenAPI specifications — API-first

Document parameter descriptions, example values, response schemas, **token usage and generation metadata**, and error conditions. Transcript publishes these for partner integration. (Task 2.1 used OpenAPI to describe tools *to an agent*; here it describes APIs *to developers*.)

Prompt Flows as a no-code builder

A new angle: Prompt Flows for **non-technical users** — the transcript has business analysts designing workflows without developer involvement. This requires business logic validation between nodes and **error paths written for non-technical readers**, not stack traces.

Multimodal input and user feedback

- Text, image and document uploads need validation, preprocessing and format conversion **before** model submission (the Task 1.3 pattern)
- **Help users understand capabilities and limits:** confidence indicators, source citations (Task 1.5), explanations of AI-generated content, and a clear distinction between **AI-generated and human-verified** content

3. Business System Enhancements

The theme: **enhance existing systems, don't replace them.**

Lambda — CRM enhancement

```
customer interaction occurs
-> event trigger -> Lambda
-> sentiment analysis
-> customer record updated with the insight
```

Transcript analyses sentiment during claim submission. If a customer is angry, that now sits in the record and the next agent is prepared. **The CRM itself was never modified — one field was populated.**

Step Functions — document processing

```
Parallel states -> different document sections concurrently
-> aggregation step
-> QUALITY THRESHOLD check
-> update downstream systems
```

The quality threshold is the new element: a poor result must not propagate downstream.

Amazon Q Business

A ready-made enterprise search and chat assistant — **you connect data, you don't build the app.**

Configuration	Guidance
Refresh schedules	Daily for frequently changing content, weekly for stable documentation
Custom document parsers	For proprietary formats
Metadata extraction	Improves relevance — transcript tags by claim type and coverage category

The metadata-for-relevance idea is the same one from Task 1.4, here inside a managed service.

Amazon Bedrock Data Automation

A managed service that extracts **structured data from unstructured content** — documents, images, audio, video.

```
triggers
-> data transformation steps
-> VALIDATION CHECKPOINTS (verify quality before proceeding)
-> notification when human review is required
```

Contrast with Task 1.3: there you assembled the pipeline yourself from Textract, Rekognition and Transcribe. **Bedrock Data Automation provides that as a managed workflow.**

Bidirectional synchronisation

- **Conflict resolution** — the AI updated a record and so did a human; which wins?
- **Audit logging of AI-driven changes** — months later, "who changed this field?" must be answerable with "the AI, using this model, at this confidence"

Measuring business impact

```
establish BASELINE metrics (before AI)
-> deploy -> track KPIs
-> statistical significance testing
```

"Processing got 20% faster" — **was that the AI, or noise?** Without a baseline you have a claim, not evidence. Same discipline as A/B testing in Task 2.4, applied at business level.

Hybrid human-AI workflows

```
Confidence threshold: 80-90% for critical decisions
```

Plus clear marking of AI-generated versus human-verified content — the adjuster's trust decision depends on knowing which is which.

4. Developer Productivity — Amazon Q Developer

Product	Audience	Purpose
Q Business	Business users	Enterprise search, internal knowledge
Q Developer	Developers	Code generation, refactoring, testing, review

Code generation with context

```
project-specific conventions
architectural patterns
dependency versions
```

Without this context Q produces generic code that doesn't match the codebase. Transcript supplies insurance domain context so generated code is compliant.

Refactoring pipelines

```
Q identifies technical debt
-> prioritised by complexity metrics + usage patterns
-> refactoring proposals with estimated effort and risk
```

Prioritisation is the point — you cannot refactor everything, so target what is both heavily used and highly complex. Transcript prioritises by performance impact.

API assistance, testing, profiling and review

Workflow	What Q does
API assistance	Analyses API usage across the codebase and suggests batching, connection pooling, better SDK patterns — the same optimisations from Task 2.4, but surfaced automatically
AI-assisted testing	Generates tests from code analysis, with coverage targets scaled to code criticality and GenAI-specific edge cases such as prompt injection
Performance profiling	Recommendations for memory management, concurrency configuration, caching
Code review automation	Evaluates PRs against best practices, security guidelines and GenAI-specific considerations

Developer knowledge base

Capture project patterns, decisions and lessons learned; index them and expose them through Q for context-aware assistance. **This is RAG applied to institutional developer knowledge** — a new engineer can ask Q about the project's conventions.

5. Advanced GenAI Applications

Largely a recap of Task 2.1. The genuinely new points:

Rate limiting in agent tool configuration

Strands tool configs include parameter validation, error handling and **rate limiting to prevent resource exhaustion during agent execution**. An agent stuck in a loop will hammer its tools — this caps it. Add it to the Task 2.1 safeguard list alongside stopping conditions, timeouts, IAM and circuit breakers.

Agent Squad supervision

Policy expert	
Damage assessor	
Fraud investigator	
Customer advocate	-> all working one commercial claim

Supervision mechanisms keep multi-agent behaviour coherent — without them four agents drift in four directions.

Validation between agent handoffs

Agent A output -> VALIDATION -> Agent B input
If Agent A treats the claim as an auto collision and Agent B treats it as property damage, both work correctly on different problems . The validation step confirms a consistent claim understanding across the handoff. Plus recovery mechanisms when an agent reaches its limits.

Prompt chaining — preserving context within token limits

Technique	Effect
Compression	Shrink the carried context
Summarization	Condense to the gist
Key information extraction	Keep only the essential facts — often best, since summaries still carry noise

Dynamic RAG

Search parameters adjust to query complexity rather than using a fixed *k*: a simple query retrieves 3 chunks, a complex one retrieves 20 and then re-ranks (Task 1.5).

Hybrid orchestration

Step Functions -> declarative: WHAT to do
EventBridge -> reactive: WHEN to start
claim submitted -> EventBridge rule -> Step Functions workflow
mid-flight fraud alert -> EventBridge -> second workflow

Combining both produces an event-driven agent system that can respond to changing conditions rather than only following a fixed script.

6. Troubleshooting Efficiency

CloudWatch Logs Insights

Query the *logs*, not just the metrics. Filter for high-latency responses, error conditions, prompt-response pair patterns, and **unexpected token usage** — a prompt suddenly consuming 10× the tokens usually means context leakage or a loop.

X-Ray with GenAI annotations

prompt complexity metrics
token counts
model parameters

These let you correlate performance problems with **specific input characteristics** — "every slow request has a long prompt" is an insight you cannot reach without them.

Q Developer error pattern recognition

Failure mode	Remediation direction
Context length errors	Compress the prompt, apply windowing
Content policy violations	Review Guardrails configuration
Hallucination patterns	Grounding checks, add retrieval

Centralised prompt registry

versioning + performance metrics
-> "which prompt version is associated with this issue?"

latency rose yesterday
-> check registry -> v8 shipped yesterday
-> v8's prompt is twice as long -> found it

This is Task 1.6's Prompt Management seen from the debugging side rather than the governance side.

Synthetic monitoring

Behaviour	
Normal monitoring	Observes real traffic — the problem is visible only once users are affected
Synthetic monitoring	Generates its own traffic with representative prompts on a schedule — detects degradation before it reaches users , and works during off-peak hours when there is no real traffic at all

Attention visualisation

The notes mention visualising attention patterns and token influence to understand why a generation occurred. **Practical caveat:** this requires access to model internals, so it applies to open-weight models — Bedrock's managed models do not expose attention weights. Treat it as a concept.

Comprehensive logging pipeline

preprocessing steps
RETRIEVAL RESULTS <- critical
model input / output
post-processing transformations

Logging retrieval results is what makes blame assignable. If the answer was wrong but the retrieved chunks were wrong first, the model behaved correctly and the fault is in retrieval. Without those logs you will tune the wrong component.